Martín Diéguez

Bureau G202, martin.dieguezlodeiro@univ-angers.fr

1 Téléchargement et découpage XML

Dans cet exercice, nous allons créer un simple lecteur de `RSS`. Télécharger le projet parseXML. Il s'agit d'un `EditText` dans lequel vous saisissez l'`URL`, et lorsque vous appuyez sur le bouton ou sur la touche Entrée, vous devrez télécharger le `XML` depuis l'`URL` indiqué, le hacher et afficher les résultats dans le `TextView` qui occupe le reste de l'écran.

- Ajouter les permissions Internet au `AndroidManifest.xml`. Complétez la méthode `actionDownload()` qui collectera l'`URL` de l'`EditText` et insérera les résultats dans le `TextView`.
- Testez avec l'URL `https://www.lemonde.fr/rss/une.xml`
- Vous devrez rechercher des balises d'éléments, et dans chacune d'elles, il y aura des balises de titre et de lien dont vous devrez ajouter le texte au résultat.
- Si vous ne recherchez pas les ouvertures et fermetures d'articles, vous verrez également le titre de la page elle-même, ce qui n'est pas vraiment une nouvelle que nous voulons montrer.
- Optionnel : affichez également la date de chaque nouvelle.

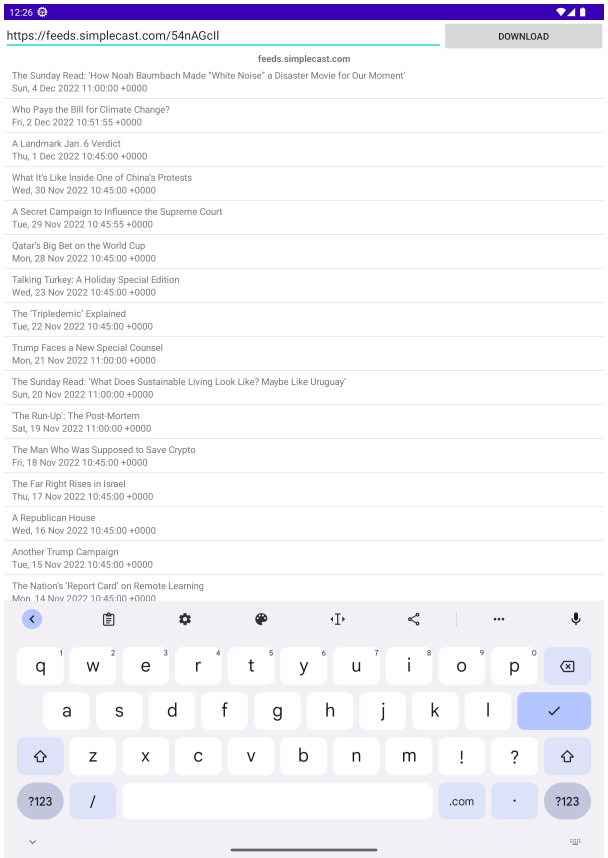
2 Lecteur RSS - version graphique

Dans cet exercice, nous allons étendre le code de l'exercice précédent. Nous allons organiser les nouvelles dans une liste d'objets News. Ensuite, nous allons créer un adaptateur pour que les nouvelles puissent être chargées dans une `ListView`. Nous en profiterons pour ajouter une image s'il y en a une. Utilisez le projet `rssReader` dans les modèles de session.

1. Créez une classe `News` avec les champs suivants :

```
private String title;
private String description;
private String link;
private String date;
private String linkImage;
private Drawable image;
```

2. Générez des `getters` et des `setters` pour chacun d'entre eux, ainsi qu'un constructeur qui les initialise à "" et met l'image à zéro.
3. Ajoutez également une méthode publique void `loadImage(String url)`.
4. Ecrivez le code qui charge dans le champ image un `BitmapDrawable` chargé à partir de l'`URL` donnée (vous pouvez utiliser un `InptuStream` obtenu à partir d'un objet de type `URL`).
5. Testez avec l'URL `https://www.lemonde.fr/rss/une.xml`
6. Copiez dans la méthode `doInBackground(...)` le code qui analyse le `XML` et remplit la news avec les champs `Titre`, `Date`, `Description`, `Link`, `LinkImage`. Notez qu'il peut y avoir différents formats `RSS`.
7. N'oubliez pas que chaque fois qu'un élément est fermé dans le `XML`, le nouvel objet news doit être ajouté à l'`ArrayList news`.



3 Service lecteur de musique

Nous allons créer un service qui démarre la lecture d'une ressource audio au démarrage et l'arrête à l'arrêt.

1. Téléchargez les projet `musicservice`. Dans le projet `MusicService`, nous avons une activité principale qui présente un bouton Start et un bouton Stop. Dans leurs `OnClickListener` respectifs, nous devrons démarrer et arrêter le service avec les méthodes `startService(...)` et `stopService(...)`, en leur passant dans les deux cas un nouvel `Intent(main, MyAudioService.class)` comme paramètre. Mais pour cela, nous devons d'abord créer la classe qui définit le service.
2. Créons une nouvelle classe Java appelée `MyAudioService` et surchargeons les méthodes `onStartCommand`, `onCreate`, `onDestroy` et `onBind`.
3. Nous déclarons un champ privé `MediaPlayer mediaPlayer`; dans la classe de service.
4. Lorsque nous démarrons le service depuis l'activité, il sera d'abord créé et la méthode `onCreate(...)` sera invoquée. C'est ici que nous créons le lecteur :

   ```
   Toast.makeText(this,"Servicio creado ...", Toast.LENGTH_LONG).show();
   mediaPlayer = MediaPlayer.create(getApplicationContext(), R.raw.ubuntu);
   mediaPlayer.setLooping(true);
   ```

   Ici, nous montrons également un Toast pour nous rassurer que le service a été lancé. La ressource `R.raw.ubuntu` est un fichier `.ogg` qui est inclus dans le dossier `res/raw` des modèles de projet (Il pourrait aussi s'agir d'un `mp3`).
5. Une fois créée, la méthode `onStartCommand(...)` sera exécutée. Il démarre la lecture et renvoie la valeur `Service.START_STICKY`.

   ```
   mediaPlayer.start() ;
   return Service.START_STICKY ;
   ```

6. Enfin, lors de la destruction du service, nous arrêterons la lecture et afficherons un `Toast` :

   ```
   Toast.makeText(this, "onDestroy : Service destroyed.", Toast.LENGTH_LONG).show() ;
   mediaPlayer.stop() ;
   ```

   Dans la méthode `onBind`, nous retournerons `null`, ce qui indique que le service n'a pas d'interface `AIDL` définie pour communiquer avec les autres. Pour que le service fonctionne dans l'application, il doit être déclaré dans le fichier `AndroidManifest.xml`, à l'intérieur de l'application.

   ```
   ...
   <service android:enabled="true" android:name=".MyAudioService"/>
   </application>
   ```

Si tout s'est bien passé, et si nous avons implémenté les listeners pour les boutons qui démarrent et arrêtent le service, cela devrait fonctionner. Essayez de démarrer le service et de quitter l'application, d'entrer dans d'autres applications, etc. La lecture du son se poursuit. Pour l'arrêter, rouvrez l'application et arrêtez-la.